

Product Mission

Pitch

Quantitative Event Engine is a Rust library and computation platform that helps equity derivatives trading operations manage the complete lifecycle of structured products and their hedges. It provides a unified event-driven architecture where the Ledger is the central abstraction—enabling real-time accounting, index calculation, research simulation, and risk aggregation through the same core components.

Users

Primary Customers

- **Structured Products Desks:** Teams managing the lifecycle of client-facing structured products (autocallables, reverse convertibles, certificates, warrants) who need real-time position tracking and P&L
- **Index Operations:** Teams creating, calculating, and rebalancing proprietary indices who need high-speed simulation for research and real-time calculation for production
- **Quantitative Researchers:** Analysts backtesting strategies, simulating portfolios, and researching new index methodologies who need maximum-speed in-memory computation

User Personas

Structured Products Trader - Role: Trader or portfolio manager on a structured products desk - **Context:** Managing lifecycle events (coupons, barriers, fixings, exercises) across a book of client positions and hedges. Generating What-If scenarios and analyzing potential systematic hedging strategies - **Pain Points:** Fragmented systems for pricing, booking, and risk. Manual reconciliation between trading and accounting. No unified view across positions, indices, and hedges. - **Goals:** Single source of truth for positions, automated lifecycle event processing, real-time P&L and risk views sliced by any dimension (desk, client, strategy, product type). Easy to generate research studies based on simple questions

Index Research Analyst - Role: Quantitative analyst designing proprietary indices - **Context:** Researching index construction rules, backtesting rebalancing strategies, simulating years of history - **Pain Points:** Slow iteration cycles, separate tools for simulation vs production, difficulty translating research into production index calculation - **Goals:** Run high-speed simulations (years of data in seconds), seamlessly promote research indices to production, unified codebase for research and live calculation. Ability to use official calculated values where these exist but also rapid prototyping in python either using REST APIs or PyO3 bindings.

Risk Manager - Role: Risk officer or portfolio risk analyst - **Context:** Needs aggregated risk views across structured products, their hedges, and index exposures - **Pain Points:** Position data scattered across systems, manual decomposition of structured products to underlying exposures, no real-time flattened risk view - **Goals:** Unified risk aggregation, automatic decomposition/flattening of complex positions, real-time exposure monitoring

The Problem

Fragmented Lifecycle Management

Equity derivatives operations struggle with disconnected systems: one for pricing, another for booking, another for risk, another for index calculation. Lifecycle events (coupons, barriers, corporate actions) require manual coordination. There's no unified view of how a client's structured product position, the desk's hedges, and the underlying index constituents relate.

Our Solution: A unified event-driven architecture where the Ledger is the central abstraction. The same event processing engine handles structured product lifecycles, index rebalancing, and hedge booking. Positions and indices are both "ledgers" that can be decomposed and flattened for unified risk views.

Research-Production Gap

Index research teams build simulations that can't easily translate to production. Different codebases, different data models, different assumptions. What works in backtesting breaks in production.

Our Solution: The same engine runs in three modes—high-speed in-memory for research, streaming for real-time calculation, persistent for production accounting. Research code IS production code, just with different persistence and speed settings.

Inflexible Accounting

Traditional ledger systems assume one booking treatment per event. But the same economic event (e.g., coupon payment) may require different ledger entries depending on legal entity, jurisdiction, accounting standard, client type, or business line.

Our Solution: A two-call pattern separating "what happened" (product-level economics) from "how to book it" (configurable move rules). Multi-dimensional context matching enables the same event to generate different ledger entries based on any combination of attributes.

Differentiators

Ledger as the Unifying Abstraction

Unlike systems that treat positions and indices as separate concepts, we model everything as a ledger. An index IS a ledger of constituents. A swap on an index IS a position that decomposes into the index's ledger. This enables unified risk views through natural decomposition/flattening.

Three-Mode Architecture

Unlike systems that force a choice between speed and durability, we support three modes from the same codebase: - **In-memory**: Maximum speed for research, Monte Carlo, backtesting - **Streaming**: Real-time calculation for live indices and P&L - **Persistent**: Durable Postgres-backed accounting for production books

Event-Driven Lifecycle Processing

Unlike batch-oriented systems, we process lifecycle events (fixings, barriers, coupons, exercises, corporate actions) through a unified event framework. The same trigger taxonomy handles structured products and indices.

Multi-Dimensional Move Rules

Unlike ledgers with hard-coded booking logic, we separate product economics from accounting treatment. Configurable move rules match on any combination of dimensions (entity, jurisdiction, product type, client type) to generate appropriate ledger entries.

Stateless Smart Contract Adapter

Unlike monolithic systems, our core processing is stateless. The Smart Contract Adapter receives all context it needs, processes it, and returns results. This enables horizontal scaling, easy testing, and clean separation of concerns.

Key Features

Core Features

- **Unified Ledger Model**: Positions and indices as ledgers with decomposition relationships
- **Event-Driven Processing**: 8 trigger types (expiry, coupon, fixing, barrier, continuous barrier, American exercise, dividends, stock splits)
- **Two-Call Pattern**: Separate product-level action calculation from position-level move generation
- **Multi-Dimensional Move Rules**: Configurable booking patterns based on context matching
- **Portfolio Tagging**: Flexible position tagging for multi-dimensional views and P&L attribution

Product Coverage

- **14 Structured Products**: Barrier Reverse Convertibles, Bonus Certificates, Warrants, Discount Certificates, Mini Certificates, Factor

Certificates, Vanilla Options, and more

- **10 Hedging Assets:** Equities, Bonds, Futures, Options, Interest Rate instruments, FX, Stock Borrow Loans

Operational Modes

- **Research Mode:** In-memory, maximum speed for simulation and backtesting
- **Production Calculation Mode:** Streaming for real-time index levels and P&L
- **Production Accounting Mode:** Postgres-backed for durable audit trail

Integration Features

- **Embedded Rust Library:** Direct integration for maximum performance
- **REST API:** Transport-agnostic callbacks, stub API for testing/demo
- **DAG Computation:** Integration with existing analytics DAG framework

Architecture Overview

